

课程笔记

veRL Agentic Loop 计算细节：异步与状态管理

基于公开课程资料整理

2025

Agent Loop 状态机 Async Rollout

视频作者/频道：五道口纳什

发布日期：2025

视频时长：约 20 分钟

项目主页：<https://github.com/hqhq1025/ai-course-notes>

目录

1 引言	2
2 为什么要异步？	2
3 状态管理	2
3.1 本章小结	2
4 总结与延伸	2

1 引言

本期回答 Agentic Loop 实现中的关键问题：为什么要异步？如何管理不同状态？

2 为什么要异步？

异步的必要性

Agentic Loop 中，不同 sample 的 Multi-turn 步数不同。如果同步等待最慢的 sample，效率极低。异步执行让已完成的 sample 先返回，最大化 GPU 利用率。

3 状态管理

Agentic Loop 中每个 sample 可能处于不同状态：

- **Generating**: 正在生成文本
- **Tool Calling**: 正在执行工具
- **Waiting**: 等待工具返回
- **Done**: 已完成所有轮次

状态转换由配置文件控制：配置了 tool 则调用 tool，配置了 interaction 则模拟用户反馈。

3.1 本章小结

异步执行和精细的状态管理是 Agentic Loop 高效运行的关键。

4 工程提醒：状态越多，越需要明确边界

Agentic Loop 的异步执行能力本质上依赖于状态机。如果状态定义不清、转换条件模糊，系统很容易在多轮工具调用里出现卡死、重复执行或错误回收资源的问题。

状态管理常见风险

- sample 迟迟无法进入 Done，导致队列泄漏
- 工具返回后没有正确切回 Generating
- 配置项过多，但转换逻辑没有统一入口

4.1 本章小结

异步系统最怕“状态说不清”。把状态边界和转换条件写清楚，比盲目追求更多并发更重要。

5 总结与延伸

1. 异步执行解决了不同 sample 步数不同的效率问题
2. 状态管理确保每个 sample 在正确的阶段执行正确的操作
3. 配置驱动的设计提供了灵活性